SLIDE 1 – INTRODUCTION

Every day, millions of images are uploaded to the internet. Humans can instantly recognize whether an image contains a cat, a dog, or a car. But how can we teach a computer to do the same thing?

This challenge lies at the heart of computer vision and deep learning.

Good morning/afternoon everyone. My name is Shradha Janib, and today I will be presenting my project, CNN Performance Evaluation System.

The focus of this project is a simple but important question:

Does making a CNN deeper always improve its performance?

To answer this, I designed and compared three CNN architectures of increasing depth and evaluated how they learn, classify images, and generalize to unseen data.

SLIDE 2 – INTRODUCTION

Most of us interact with CNNs every day without even realizing it.

When your smartphone unlocks using facial recognition, when a self-driving car detects pedestrians and road signs, or when doctors use AI to analyse medical scans, CNNs are often working behind the scenes.

What makes CNNs powerful is their ability to automatically discover important patterns within images rather than relying on manually designed features.

However, one of the biggest design decisions when building a CNN is determining how deep the network should be.

While deeper networks can learn more sophisticated features, they also require more computational resources and may become harder to train.

This project explores whether increasing network depth genuinely improves image classification performance.

SLIDE 3 – AIM

To investigate whether deeper CNNs truly perform better, I compared three architectures of increasing depth.

Rather than looking at accuracy alone, I evaluated the models from multiple perspectives.

These included how accurately they classified unseen images, how efficiently they learned during training, which classes they classified correctly or incorrectly, and whether increased complexity resulted in meaningful performance improvements.

SLIDE 4 – DATASET STRUCTURE

For this investigation, I used the CIFAR-10 dataset, one of the most widely used benchmark datasets in image classification research.

The dataset contains 60,000 colour images distributed across 10 object categories, including airplanes, automobiles, birds, cats, deer, dogs, frogs, horses, ships, and trucks.

Each image has a resolution of 32 by 32 pixels.

The dataset is divided into 50,000 training images and 10,000 testing images.

The training set is used to teach the CNN models to recognize visual patterns, while the testing set evaluates how well the models generalize to unseen images.

Another important advantage of CIFAR-10 is its balanced class distribution, which ensures a fair comparison between different CNN architectures.

SLIDE 5 – IMPLEMENTATION STEPS

Building an effective CNN involves much more than simply training a model and checking its accuracy.

To answer the research question systematically, I followed a structured six-step implementation process.

First, data preprocessing and normalization were performed.

Second, the dataset was loaded using PyTorch DataLoaders.

Third, three CNN architectures of different depths were created.

Fourth, the models were trained on the CIFAR-10 dataset.

Fifth, performance evaluation was conducted using test accuracy.

Finally, confusion matrix analysis was used to investigate class-wise prediction performance and common classification errors.

SLIDE 6 – DATA PROCESSING AND DATASET LOADING

Now that we have established the overall implementation framework, let's begin with the foundation of every successful deep learning project: the data.

No matter how powerful a CNN architecture is, its performance depends heavily on how well the data is prepared before training.

The CIFAR-10 images were first converted into PyTorch tensors using the ToTensor() function, allowing the CNN to process image data mathematically.

Next, the pixel values were normalized from a range of 0 to 255 to approximately minus 1 to plus 1.

This improves training stability, speeds up learning, and helps the models converge more effectively.

The dataset was then loaded using PyTorch DataLoaders, which process the data in batches rather than loading all images at once.

A batch size of 64 was selected to balance memory usage and computational efficiency.

Finally, the training data was shuffled before each epoch to improve learning and prevent the model from developing unwanted patterns based on data order.

These preprocessing steps ensured efficient and reliable CNN training.

SLIDE 7 – CNN MODEL CREATION

Imagine teaching three students the same subject.

One receives only the basics, another receives more detailed instruction, and the third receives the most advanced training.

Which student would perform best?

To answer a similar question for neural networks, I designed three CNN architectures of increasing depth.

The Shallow CNN contains 2 convolutional layers and focuses on learning basic visual features such as edges and textures.

The Medium CNN contains 4 convolutional layers, enabling it to capture more detailed patterns and image representations.

The Deep CNN contains 6 convolutional layers and includes dropout regularization, allowing it to learn complex hierarchical features while reducing overfitting.

All three architectures use convolutional layers for feature extraction, ReLU activation functions to introduce non-linearity, MaxPooling layers to reduce image dimensions, and fully connected layers for final classification into the 10 CIFAR-10 classes.

SLIDE 8 – MODEL TRAINING

Designing a CNN is like building a race car—but its true performance is only revealed once it enters the race.

Similarly, after creating the CNN architectures, the next step was to train them on the CIFAR-10 dataset and observe how effectively they could learn from the images.

To optimize learning, I used the Adam Optimizer, which efficiently updates network parameters and helps the models converge faster.

Cross-Entropy Loss was used to measure prediction errors and guide the optimization process for multi-class classification.

Each model was trained for 10 epochs, meaning the entire training dataset was processed ten times.

During forward propagation, images passed through the network to generate predictions.

The prediction error was then calculated, and backpropagation was used to compute gradients and update the model weights.

Throughout training, both accuracy and loss were monitored to evaluate learning effectiveness and convergence.

SLIDE 9 – TRAINING ACCURACY COMPARISON

Now let's examine how effectively the models learned from the training data.

As shown in the graph, all three CNN architectures showed a steady increase in training accuracy across the 10 epochs, indicating successful learning and convergence.

The Medium CNN achieved the highest training accuracy of 96.49%, suggesting that it provided an effective balance between network complexity and feature learning capability.

The Shallow CNN achieved 93.58%, demonstrating strong performance but with limitations in learning more complex image features.

The Deep CNN achieved 87.18%. Although it contained more layers, dropout regularization and increased complexity resulted in slower convergence during training.

An important observation is that higher model complexity did not automatically result in higher training accuracy.

SLIDE 10 – TRAINING LOSS COMPARISON

Training loss provides a deeper insight into the learning process by measuring how far predictions are from the correct answers.

All three models showed a consistent decrease in loss throughout training, indicating effective learning and optimization.

The Medium CNN achieved the lowest final loss of 0.0985, demonstrating the most efficient learning process.

The Shallow CNN achieved a final loss of 0.1864, while the Deep CNN recorded the highest final loss of 0.3621.

The Deep CNN's higher loss was largely due to its increased complexity and the use of dropout regularization, which intentionally makes learning more challenging to improve generalization.

SLIDE 11 – PERFORMANCE EVALUATION

At this point, you might expect the Medium CNN to be the best-performing model because it achieved both the highest training accuracy and the lowest training loss.

However, machine learning often teaches us an important lesson:

A model that performs best during training does not always perform best in the real world.

When evaluated on unseen test images, the Deep CNN achieved the highest test accuracy of 77.57%.

The Medium CNN achieved 75.14%, while the Shallow CNN achieved 72.67%.

The additional convolutional layers and dropout regularization enabled the Deep CNN to learn richer and more transferable image features, resulting in stronger generalization performance.

This was the most important finding of the project.

SLIDE 12 – CONFUSION MATRIX ANALYSIS

While test accuracy tells us how often the model is correct, the confusion matrix helps us understand where the model succeeds and where it struggles.

The Deep CNN performed particularly well on classes such as car, ship, frog, and truck, achieving the highest numbers of correct predictions.

However, visually similar classes such as cat and dog, and deer and horse, remained more challenging.

These misclassifications occurred because the classes share similar shapes, textures, and visual characteristics.

This shows that the model learned meaningful image features, but visually similar categories remain one of the biggest challenges in image classification.

SLIDE 13 – RESULT SUMMARY

Let's return to the question that motivated this project:

Does increasing CNN depth improve image classification performance?

Based on the results, the answer is yes—but not always in the way we might expect.

Although the Medium CNN achieved the highest training accuracy and lowest training loss, it was the Deep CNN that achieved the highest test accuracy of 77.57%.

This demonstrates that learning the training data well is not enough.

The true goal of machine learning is to perform effectively on data the model has never seen before.

The Deep CNN's additional layers and dropout regularization enabled it to learn more robust and transferable image features, resulting in stronger generalization performance.

The key takeaway is simple:

The best model is not the one that memorizes the data, but the one that generalizes the best.

SLIDE 14 – REFERENCES

Before concluding, I would like to acknowledge the key resources that supported this project.

The work of LeCun and colleagues provided the theoretical foundation for Convolutional Neural Networks and demonstrated how convolutional and pooling layers automatically learn image features.

Krizhevsky and colleagues demonstrated the effectiveness of deep CNN architectures for image classification and helped establish deep learning as a dominant approach in computer vision.

The CIFAR-10 dataset served as the benchmark dataset used to train and evaluate all CNN models.

Finally, the PyTorch framework and documentation provided the tools required to design, train, and evaluate the CNN architectures presented in this study.

Thank you for your time and attention. I would be happy to answer any questions.